MODULE 2

PROCESS MANAGEMENT

Process Concept

- ✓ A process is a **program in execution**.
- ✓ It also includes (Figure below):

1) **Program Counter** to indicate the current activity.
2) **Registers Content** of the processor.
3) **Process Stack** contains temporary data.
4) **Data Section** contains global variables.
5) **Heap** is memory that is dynamically allocated during process run time.

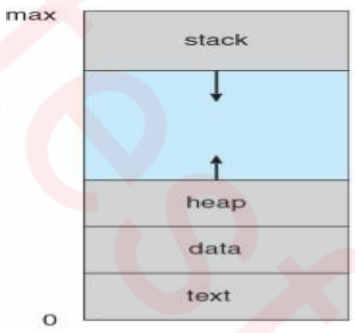


Figure: Process in memory

- ✓ A program by itself is not a process.
  1) A process is an active-entity.
  2) A program is a passive-entity such as an executable-file stored on disk.
- ✓ A program becomes a process when an executable-file is loaded into memory.

Process State

- ✓ As a process executes, it changes state. The **state of a process** is defined in part by the **current activity of that process**. Each process may be in one of the following states:
  - **New:** The process is being created.
  - **Running:** Instructions are being executed.
  - **Waiting**: The process is waiting for some event to occur (such as an I/O completion or reception of a signal).
  - **Ready**: The process is waiting to be assigned to a processor.
  - **Terminated:** The process has finished execution.

vtunotesforall.in

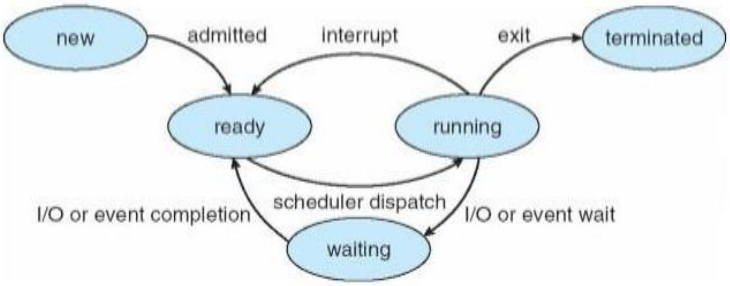


Figure: Diagram of process state.

Process Control Block (PCB)

- ✓ Each process is represented in the operating system by a **process control block (PCB)**—also called a **task control block**. A PCB is shown in Figure.
- ✓ It contains many pieces of information associated with a specific process, including these:

1. **Process state**. The state may be new, ready, running, and waiting, halted, and so on.
2. **Program counter**. The counter indicates the address of the next instruction to be executed for this process.
3. **CPU registers**. They include *accumulators, index registers, stack pointers, and general-purpose registers*, plus any condition-code information.
4. **CPU-scheduling information**. This information includes a process priority, pointers to scheduling queues, and any other scheduling parameters.
5. **Memory-management information**. This information may include such information as the value of the base and limit registers, the page tables, or the segment tables, depending on the memory system used by the operating system.
6. **Accounting information**. This information includes the amount of CPU and real time used, time limits, account numbers, job or process numbers, and so on.
7. **I/O status information.** This information includes the list of I/O devices allocated to the process, a list of open files, and so on.

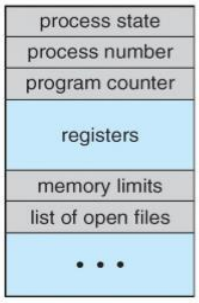


Figure: Process control block (PCB).

vtuprofessorali.in

Process Scheduling

- ✓ The objective of multiprogramming is to have some process running at all times, to maximize **CPU utilization**.

Scheduling Queues

- ✓ As processes enter the system, they are put into a ***job queue***, *which consists of all processes in the system.*
- ✓ *The processes that are* ***residing in main memory*** *and* ***are ready and waiting to execute*** *are kept on a list called the* ***ready queue***. This queue is generally stored as a linked list.
- ✓ *A ready-queue header contains pointers to the first and final PCBs* in the list. *Each PCB includes a pointer field that points to the next PCB in the ready queue*. The system also includes other queues.
- ✓ When a process is allocated the CPU, it executes for a while and eventually quits, is interrupted, or waits for the occurrence of a particular event, such as the completion of an I/O request.

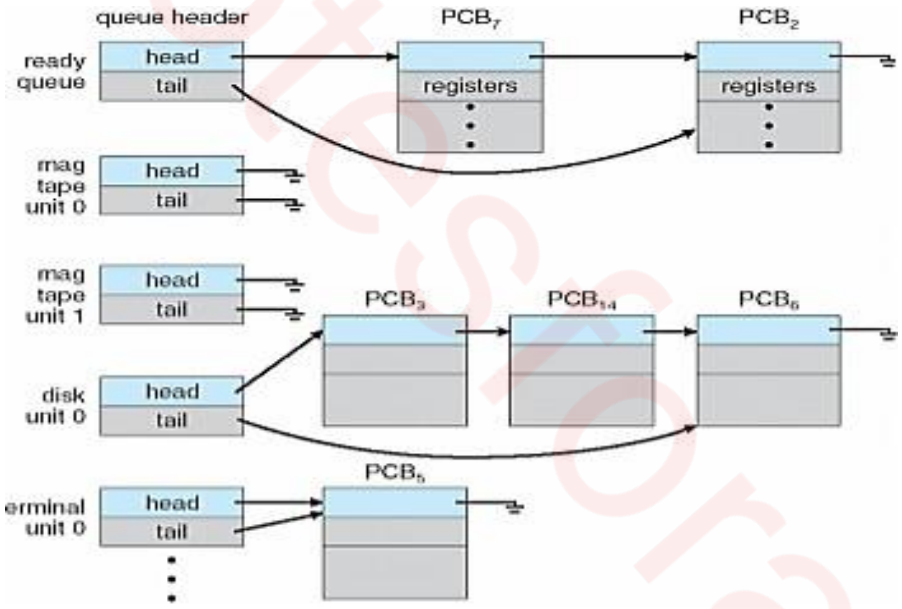


Figure: the ready queue and various I/O device queues

- ✓ A common representation of a process scheduling is a queuing diagram, as shown in Figure below. Each rectangular box represents a queue.
- ✓ Two types of queues are present: the **ready queue** and a set of **device queues**. The circles represent the resources that serve the queues, and the arrows indicate the flow of processes in the system.
- ✓ A new process is initially put in the ready queue. It waits there until it is selected for execution, or is dispatched. Once the process is allocated the CPU and is executing, one of several events could occur:
  - The process could issue an I/O request and then be placed in an I/O queue.
  - The process could create a new sub-process and wait for the sub-process's termination.

- The process could be removed forcibly from the CPU, as a result of an interrupt, and be put back in the ready queue.

✓ In the first two cases, the process eventually switches from the waiting state to the ready state and is then put back in the ready queue. A process continues this cycle until it terminates, at which time it is removed from all queues and has its PCB and resources de-allocated.

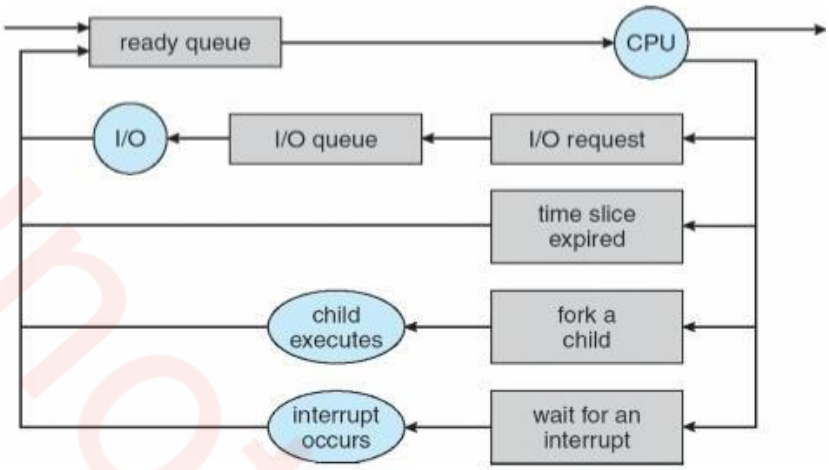


Figure: Queuing-diagram representation of process scheduling.

Schedulers

✓ Three types of schedulers:

1) Long-term scheduler
2) Short-term scheduler and
3) Medium-term schedulers

Long-Term Scheduler	Short-Term Scheduler
Also called job scheduler.	Also called CPU scheduler.
Selects which processes should be brought into the ready-queue.	Selects which process should be executed next and allocates CPU.
Need to be invoked only when a process leaves the system and therefore executes much less frequently.	Need to be invoked to select a new process for the CPU and therefore executes much more frequently.
May be slow, minutes may separate the creation of one new process and the next.	Must be fast, a process may execute for Only a few milliseconds.
Controls the degree of multiprogramming.	

✓ Processes can be described as either:

1) I/O-bound Process

- Spends more time doing I/O operation than doing computations.

- Many short CPU bursts.

2) CPU-bound Process

- Spends more time doing computations than doing I/O operation.
- Few very long CPU bursts.

✓ Why long-term scheduler should select a good process mix of I/O-bound and CPU-bound processes ?

Ans: 1) If all processes are I/0 bound, then
   i) Ready-queue will almost always be empty, and
   ii) Short-term scheduler will have little to do.
2) If all processes are CPU bound, then
   i) I/0 waiting queue will almost always be empty (devices will go unused) and System will be unbalanced.

✓ Some operating systems, such as time-sharing systems, may introduce an additional, intermediate level of scheduling. This medium-term scheduler is diagrammed in Figure.

✓ The key idea behind a medium-term scheduler is that **sometimes it can be advantageous to remove processes from memory** (and from active contention for the CPU) and thus **reduce the degree of multiprogramming.**

✓ Later, the process can be reintroduced into memory, and its execution can be continued where it left off. This scheme is called **swapping**. The process is swapped out, and is later swapped in, by the medium-term scheduler.

✓ Swapping may be necessary to improve the process mix or because a change in memory requirements has overcommitted available memory, requiring memory to be freed up.

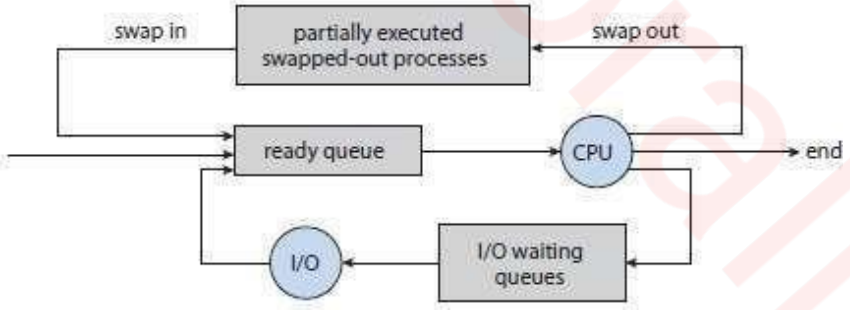


Figure: Addition of medium-term scheduling to the queuing diagram

Context Switch

✓ Context-switch means *saving the state of the old process and switching the CPU to another process.*

✓ The context of a process is represented in the PCB of the process; it includes

- *value of CPU registers*
- *process-state and*
- *Memory-management information.*

✓ Disadvantages:

1) Context-switch time is pure overhead, because the system does no useful work while switching.
2) Context-switch times are highly dependent on hardware support.

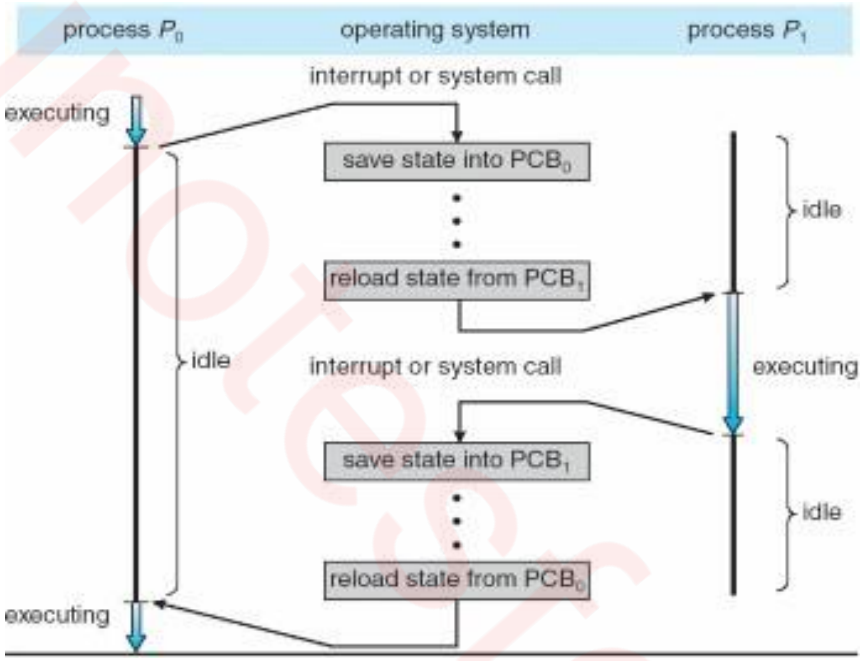


Figure: Diagram showing CPU switch from process to process.

Operations on Processes

1) Process Creation and
2) Process Termination

Process Creation

✓ A process may create several new processes, via a create-process system call, during the course of execution.

✓ The creating process is called a ***parent process***, and the *new processes are called the children of that process.*

✓ Operating systems identify processes according to a unique **process identifier** (or **pid**), which is typically an **integer number**.

✓ A process will need certain resources (CPU time, memory, files, I/O devices) to accomplish its task.

- ✓ Child-process may
  - get resources directly from the OS or
  - get resources of parent-process. This prevents any process from overloading the system

- ✓ Two options exist when a process creates a new process:
  1) The parent & the children execute concurrently.
  2) The parent waits until all the children have terminated.

- ✓ Two options exist in terms of the address-space of the new process:
  1) The child-process is a duplicate of the parent-process (it has the same program and data as the parent).
  2) The child-process has a new program loaded into it.

Process creation in UNIX

- ✓ In UNIX, each process is identified by its process identifier (pid), which is a unique integer.
- ✓ A new process is created by the **fork()** system-call (Figure 1.29 & 1.30).
- ✓ The new process consists of a copy of the address-space of the original process.
- ✓ Both the parent and the child continue execution with one difference:
  1) The return value for the fork() is
     zero for the new (child) process.
  2) The return value for the fork() is
     nonzero pid of the child for the parent-process.

- ✓ Typically, the **exec()** system-call is used after a fork() system-call by one of the two processes to replace the process's memory-space with a new program.
- ✓ The parent can issue **wait()** system-call to move itself off the ready-queue.

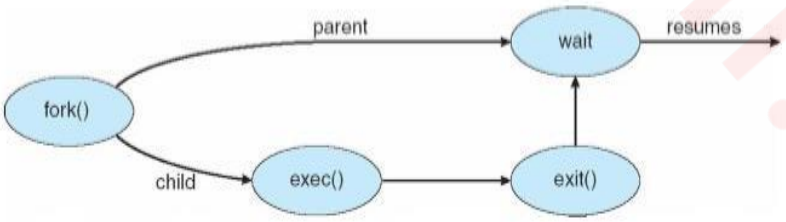


Figure : Process creation.

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
       fprintf(stderr, "Fork Failed");
       return 1;
    }
    else if (pid == 0) { /* child process */
       execlp("/bin/ls","ls",NULL);
    }
    else { /* parent process */
       /* parent will wait for the child to complete */
       wait(NULL);
       printf("Child Complete");
    }

    return 0;
}
```

Process Termination

- ✓ A process terminates when it executes the last statement (in the program).
- ✓ Then, the OS deletes the process by using **exit()** system-call.
- ✓ Then, the OS de-allocates all the resources of the process. The resources include
  → memory
  → open files and
  → I/0 buffers.
- ✓ Process termination can occur in following cases:
  → A process can cause the termination of another process via **TerminateProcess()** system-call.
  → Users could arbitrarily **kill** the processes.
- ✓ A parent terminates the execution of children for following reasons:
  3) The child has exceeded its usage of some resources.
  4) The task assigned to the child is no longer required.
  5) The parent is exiting, and the OS does not allow a child to continue.
- ✓ In some systems, if a process terminates, then all its children must also be terminated. This phenomenon is referred to as **cascading termination**.

vtunotesforall.in

Interposes Communication(IPC)

✓ Processes executing concurrently in the OS may be 1) Independent processes or
2) Co-operating processes.

1) A process is **independent** if
   i) The process cannot affect or be affected by the other processes.
   ii) The process does not share data with other processes.
2) A process is **co-operating** if
   i) The process can affect or be affected by the other processes.
   ii) The process shares data with other processes.

✓ Advantages of process co-operation:

1) Information Sharing

- Since many users may be interested in same piece of information (ex: shared file).

2) Computation Speedup

- We must break the task into subtasks.
- Each subtask should be executed in parallel with the other subtasks.
- The speed can be improved only if computer has multiple processing elements such as
  → CPUs or
  → I/O channels.

3) Modularity

- Divide the system-functions into separate processes or threads.

4) Convenience

- An individual user may work on many tasks at the same time.
- For ex, a user may be editing, printing, and compiling in parallel.

✓ Two basic models of IPC (Figure below): 1) Shared-memory and
2) Message passing.

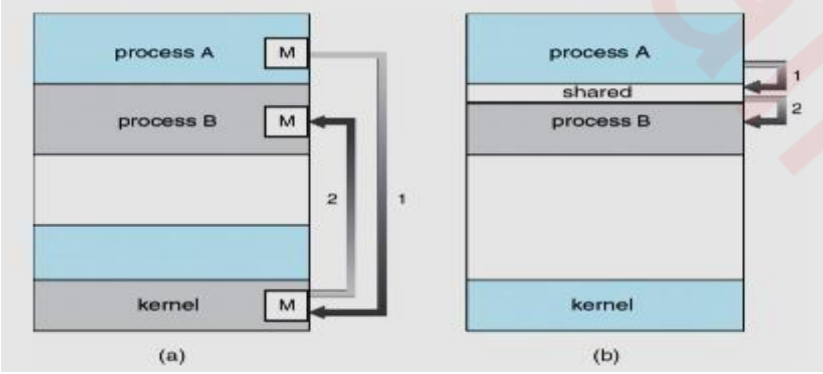


Figure: Communications models, (a) Message passing, (b) Shared memory

1. Shared-Memory Systems

- ✓ Typically, a shared-memory region resides in the address space of the process creating the shared-memory segment.
- ✓ To illustrate the concept of cooperating processes, let's consider the **producer-consumer problem**.
- ✓ **A producer process produces information that is consumed by a consumer process**.
- ✓ One **solution to the producer-consumer problem uses shared memory**. To allow producer and consumer processes to run **concurrently**, we must have available a **buffer** of items that can be filled by the producer and emptied by the consumer.
- ✓ This buffer will reside in a region of memory that is shared by the producer and consumer processes.

  A producer can produce one item while the consumer is consuming another item. So that the consumer does not try to consume an item that has not yet been produced.
- ✓ Two types of buffers can be used:
  1) **Unbounded-Buffer** places no practical limit on the size of the buffer.
  2) **Bounded-Buffer** assumes that there is a fixed buffer-size.
- ✓ Advantages:
  1) Allows maximum speed and convenience of communication.
  2) Faster.

2. Message-Passing Systems

- ✓ Message passing provides a mechanism to allow processes to communicate and to synchronize their actions without sharing the same address space and is particularly useful in a distributed environment, where the communicating processes may reside on different computers connected by a network.
- ✓ A message-passing facility provides at least two operations:
  - send(message) and
  - **receive**(message)
- ✓ Messages sent by a process can be of either
  - fixed or
  - variable size.
- ✓ If only fixed-sized messages can be sent, the system-level implementation is straightforward.
- ✓ Variable-sized messages require a more complex system-level implementation, but the programming task becomes simpler.
- ✓ A communication-link must exist between processes to communicate
- ✓ Three methods for implementing a link:

vtuprofessionalli.in

1) Direct or indirect communication.
2) Symmetric or asymmetric communication.
3) Automatic or explicit buffering.

- ✓ Two operations:
  1) send(P,message): Send a message to process P.
  2) receive(Q,message): Receive a message from process Q.
- ✓ Advantages:
  1) Useful for exchanging smaller amounts of data („.“ No conflicts need beavoided).
  2) Easier to implement.
  3) Useful in a distributed environment.

Naming (Direct and Indirect)

- ✓ Processes that want to communicate must have a way to refer to each other. They can use either **direct** or **indirect** communication.
- ✓ **In direct communication, each process that wants to communicate must explicitly name the recipient or sender of the communication**. In this scheme, the send() and receive() primitives are defined as:

  • send(P, message)—Send a message to process P.

  • receive (Q, message)—Receive a message from process Q.
- ✓ A communication link in this scheme has the following properties:

  • A link is established automatically between every pair of processes that want to communicate. The processes need to know only each other's identity to communicate.

  • A link is associated with exactly two processes.

  • Between each pair of processes, there exists exactly one link.
- ✓ This scheme exhibits **symmetry** in addressing; that is, both the sender process and the receiver process must name the other to communicate.
- ✓ A variant of this scheme employs **asymmetry** in addressing. Here, only the sender names the recipient; the recipient is not required to name the sender.

  • send(P, message)—Send a message to process P.

  • receive(id, message)—-Receive a message from any process; the variable id is set to the name of the process with which communication has taken place.
- ✓ The **disadvantage** in both of these schemes is the limited modularity of the resulting process definitions. Changing the identifier of a process may necessitate examining all other process definitions.
- ✓ **Indirect communication**: **the messages are sent to and received from mailboxes, or ports**. A mailbox can be viewed abstractly as an object into which messages can be placed by processes and from which messages can be removed.

- ✓ Each mailbox has a unique identification. a process can communicate with some other process via a number of different mailboxes. Two processes can communicate only if the processes have a shared mailbox, however.
- ✓ The send () and receive () primitives are defined as follows:
  • send(A, message)—Send a message to mailbox A.

  • receive(A, message)—Receive a message from mailbox A.
- ✓ In this scheme, a communication link has the following properties:
  • A link is established between a pair of processes only if both members of the pair have a shared mailbox.

  • A link may be associated with more than two processes.

  • Between each pair of communicating processes, there may be a number of different links, with each link corresponding to one mailbox.
- ✓ A mailbox may be owned either by a process or by the operating system. If the mailbox is owned by a process (that is, the mailbox is part of the address space of the process), then we distinguish between the owner (who can only receive messages through this mailbox) and the user (who can only send messages to the mailbox).

Synchronization.

- ✓ Message passing may be either **blocking** or **non-blocking**— also known as **synchronous** and **asynchronous**.
  • **Blocking send.** The sending process is blocked until the message is received by the receiving process or by the mailbox.

  • **Non-blocking send**. The sending process sends the message and resumes operation.

  • **Blocking receive**. The receiver blocks until a message is available.

  • **Non-blocking receive**. The receiver retrieves either a valid message or a null.
- ✓ When both send() and receive() are blocking, we have a rendezvous between the sender and the receiver. The solution to the producer-consumer problem becomes trivial when we use blocking send() and receive0 statements.
- ✓ The producer merely invokes the blocking send() call and waits until the message is delivered to either the receiver or the mailbox. Likewise, when the consumer invokes receive (), it blocks until a message is available.

Buffering

- ✓ Whether communication is direct or indirect, messages exchanged by communicating processes reside in a **temporary queue**. Basically, such queues can be implemented in three ways:

  • **Zero capacity.** The queue has a maximum length of **zero**; thus, the link cannot have any messages waiting in it. In this case, the sender must block until the recipient receives the message.

Vtuneseries.in

• **Bounded capacity**. The queue has **finite length** n; thus, at most n messages can reside in it. If the queue is not full when a new message is sent, the message is placed in the queue (either the message is copied or a pointer to the message is kept), and the sender can continue execution without waiting. The links capacity is finite, however. If the link is full, the sender must block until space is available in the queue.

• **Unbounded capacity**. The queues length is potentially **infinite**; thus, any number of messages can wait in it. The sender never blocks.

MODULE 2 CHAPTER 2

MULTI-THREADED PROGRAMMING

Multi-Threaded Programming

- A thread is a basic unit of CPU utilization.
- It consists of

  → thread ID

  → PC

  → register-set and

  → stack.
- It shares with other threads belonging to the same process its code-section & data-section.
- A traditional (or heavy weight) process has a single thread of control.
- If a process has multiple threads of control, it can perform more than one task at a time. Such a process is called **multi-threaded process.**

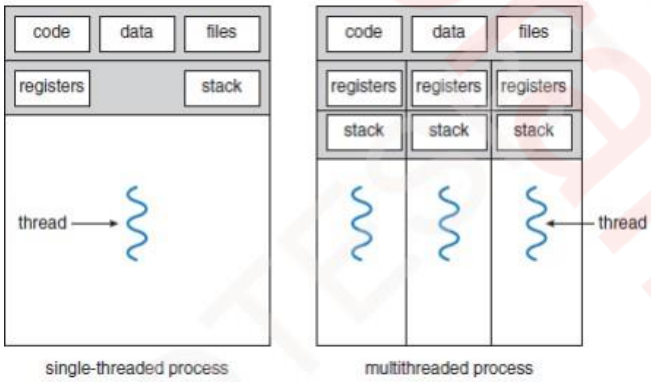


Figure 2.1 Single-threaded and multithreaded processes

Motivation

1. The software-packages that run on modern PCs are multithreaded.

   An application is implemented as a separate process with several threads of control.

For ex: A word processor may have

→ first thread for displaying graphics

→ second thread for responding to keystrokes and

→ third thread for performing grammar checking.

2. In some situations, a single application may be required to perform several similar tasks.

   For ex: A web-server may create a separate thread for each client request.

   This allows the server to service several concurrent requests.

3. RPC servers are multithreaded.

   When a server receives a message, it services the message using a separate thread.

   This allows the server to service several concurrent requests.

4. Most OS kernels are multithreaded;

   Several threads operate in kernel, and each thread performs a specific task, such as

   → managing devices or

   →interrupt handling.

Benefits

1) **Responsiveness**
   - A program may be allowed to continue running even if part of it is blocked. Thus, increasing responsiveness to the user.
2) **Resource Sharing**
   - By default, threads share the memory (and resources) of the process to which they belong. Thus, an application is allowed to have several different threads of activity within the same address-space.
3) **Economy**
   - Allocating memory and resources for process-creation is costly. Thus, it is more economical to create and context-switch threads.
4) **Utilization of Multiprocessor Architectures**
   - In a multiprocessor architecture, threads may be running in parallel on different processors. Thus, parallelism will be increased.

Multi-Threading Models

- Support for threads may be provided at either

  1) The user level, for user threads or

  2) By the kernel, for kernel threads.

vtunotesforall.in

- User-threads are supported above the kernel and are managed without kernel support. Kernel-threads are supported and managed directly by the OS.
- Three ways of establishing relationship between user-threads & kernel-threads:
  1) Many-to-one model
  2) One-to-one model and
  3) Many-to-many model

1. Many-to-One Model

- Many user-level threads are mapped to one kernel thread (Figure 2.2)
- Advantage:
  1) Thread management is done by the thread library in user space, so it is efficient.
- Disadvantages:
  1) The entire process will block if a thread makes a blocking system-call.
  2) Multiple threads are unable to run in parallel on multiprocessors.
- For example:
  → Solaris green threads
  → GNU portable threads

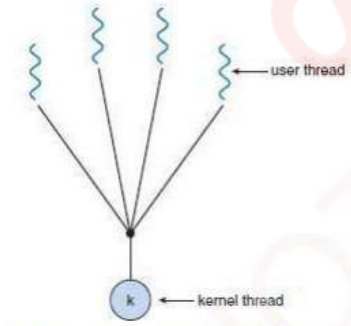


Figure 2.2 Many-to-one model

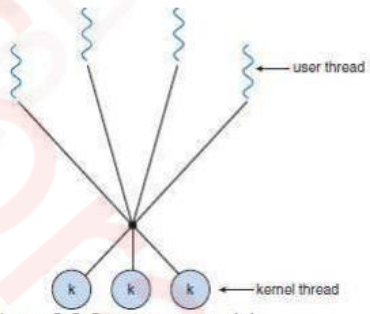


Figure 2.3 One-to-one model

2. One-to-One Model

- Each user thread is mapped to a kernel thread (Figure 2.3).
- Advantages:
  1) It provides more concurrency by allowing another thread to run when a thread makes a blocking system-call.
  2) Multiple threads can run in parallel on multiprocessors.
- Disadvantage:
  1) Creating a user thread requires creating the corresponding kernel thread.

  For example:

→ Windows NT/XP/2000

→ Linux.

3. Many-to-Many Model

- Many user-level threads are multiplexed to a smaller number of kernel threads (Figure 2.4).
- Advantages:

  1) Developers can create as many user threads as necessary

  2) The kernel threads can run in parallel on a multiprocessor.

  3) When a thread performs a blocking system-call, kernel can schedule another thread for execution.

Two Level Model

- A variation on the many-to-many model is the two level-model (Figure 2.5).
- Similar to M:M, except that it allows a user thread to be bound to kernel thread.
- For example:

  → HP-UX

  → Tru64 UNIX.

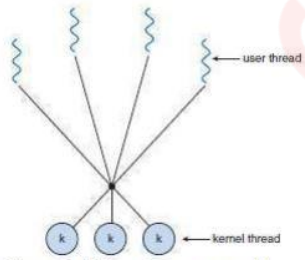


Figure 2.4 Many-to-many model

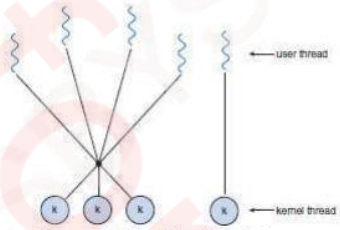


Figure 2.5 Two-level model

Thread Libraries

- It provides the programmer with an API for the creation and management of threads.
- Two ways of implementation:

  1) First Approach

  - Provides a library entirely in user space with no kernel support.
  - All code and data structures for the library exist in the user space.

  2) Second Approach

  - Implements a kernel-level library supported directly by the OS.
  - Code and data structures for the library exist in kernel space.

- Three main thread libraries:
  1) POSIX Pthreads
  2) Win32 and
  3) Java.

Pthreads

- This is a POSIX standard API for thread creation and synchronization.
- This is a specification for thread-behaviour, not an implementation.
- OS designers may implement the specification in any way they wish.
- Commonly used in: UNIX and Solaris.

Java Threads

- Threads are the basic model of program-execution in → Java program and → Java language.
- The API provides a rich set of features for the creation and management of threads.
- All Java programs comprise at least a single thread of control.
- Two techniques for creating threads:
  1) Create a new class that is derived from the Thread class and override its run() method.
  2) Define a class that implements the Runnable interface. The Runnable interface is defined as follows:

```
public interface Runnable
{
   public abstract void run();
}
```

Threading Issues

1. fork() and exec() System-calls

- fork () is used to create a separate, duplicate process.
- If one thread in a program calls fork (), then
  1) Some systems duplicate all threads and
  2) Other systems duplicate only the thread that invoked the fork ().
- If a thread invokes the exec (), the program specified in the parameter to exec () will replace the entire process including all threads.

vtunotesforall.in

2. Thread Cancellation

- This is the task of terminating a thread before it has completed.
- Target thread is the thread that is to be cancelled
- Thread cancellation occurs in two different cases:

  1) Asynchronous cancellation: One thread immediately terminates the target thread.

  2) Deferred cancellation: The target thread periodically checks whether it should be terminated.

3. Signal Handling

- In UNIX, a signal is used to notify a process that a particular event has occurred.
- All signals follow this pattern:

  1) A signal is generated by the occurrence of a certain event.

  2) A generated signal is delivered to a process.

  3) Once delivered, the signal must be handled.
- A signal handler is used to process signals.
- A signal may be received either synchronously or asynchronously, depending on the source. 1) Synchronous signals ¬ Delivered to the same process that performed the operation causing the signal. ¬ E.g. illegal memory access and division by 0.

  3) Asynchronous signals ¬ Generated by an event external to a running process. ¬ E.g. user terminating a process with specific keystrokes.
- Every signal can be handled by one of two possible handlers:

  1) A Default Signal Handler ¬ Run by the kernel when handling the signal.

  2) A User-defined Signal Handler ¬ Overrides the default signal handler.
- In single-threaded programs, delivering signals is simple. In multithreaded programs, delivering signals is more complex. Then, the following options exist:

  1) Deliver the signal to the thread to which the signal applies.

  2) Deliver the signal to every thread in the process.

  3) Deliver the signal to certain threads in the process.

  4) Assign a specific thread to receive all signals for the process.

4. Thread Pools

- The basic idea is to

  → create a no. of threads at process-startup and

  → place the threads into a pool (where they sit and wait for work)
- Procedure:

vtuportal.in

1) When a server receives a request, it awakens a thread from the pool.

2) If any thread is available, the request is passed to it for service.

3) Once the service is completed, the thread returns to the pool.

- Advantages:

1) Servicing a request with an existing thread is usually faster than waiting to create a thread.

2) The pool limits the no. of threads that exist at any one point.

- No. of threads in the pool can be based on factors such as

→ no. of CPUs

→ amount of memory and

→ expected no. of concurrent client-requests.

vtunotesforall.in

PROCESS SCHEDULING

Basic Concepts

- In a single-processor system,
  → only one process may run at a time.
  → other processes must wait until the CPU is rescheduled.
- Objective of multiprogramming:
  → to have some process running at all times, in order to maximize CPU utilization.

CPU-I/0 Burst Cycle

- Process execution consists of a cycle of → CPU execution and → I/O wait (Figure 2.6 & 2.7).
- Process execution begins with a CPU burst, followed by an I/O burst, then another CPU burst, etc…
- Finally, a CPU burst ends with a request to terminate execution.
- An I/O-bound program typically has many short CPU bursts. A CPU-bound program might have a few long CPU bursts.

vtunotesforall.in

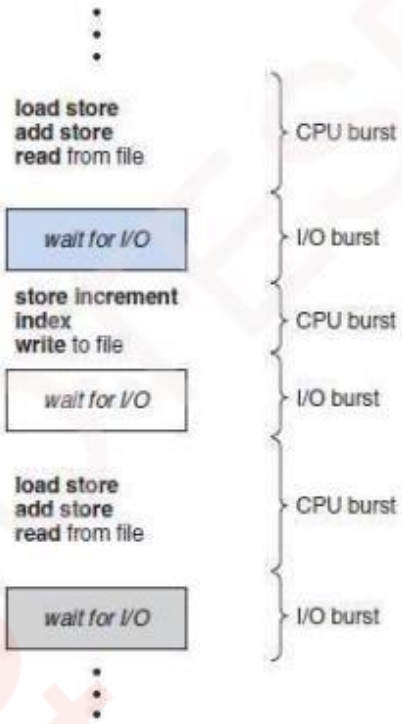


Fig: Alternating sequence of CPU and I/O bursts

CPU Scheduler

This scheduler

→ selects a waiting-process from the ready-queue and

→ allocates CPU to the waiting-process.

The ready-queue could be a FIFO, priority queue, tree and list.

The records in the queues are generally process control blocks (PCBs) of the processes.

CPU Scheduling

- Four situations under which CPU scheduling decisions take place:
  1) When a process switches from the running state to the waiting state. For ex; I/O request.
  2) When a process switches from the running state to the ready state. For ex: when an interrupt occurs.
  3) When a process switches from the waiting state to the ready state. For ex: completion of I/O.
  4) When a process terminates.
- Scheduling under 1 and 4 is non-preemptive.

- Scheduling under 2 and 3 is preemptive.

Non Preemptive Scheduling

- Once the CPU has been allocated to a process, the process keeps the CPU until it releases the CPU either
  → by terminating or
  → by switching to the waiting state.

Preemptive Scheduling

- This is driven by the idea of prioritized computation.
- Processes that are runnable may be temporarily suspended
- Disadvantages:
  1) Incurs a cost associated with access to shared-data.
  2) Affects the design of the OS kernel.

Dispatcher

- It gives control of the CPU to the process selected by the short-term scheduler.
- The function involves:
  1) Switching context
  2) Switching to user mode &
  3) Jumping to the proper location in the user program to restart that program.
- It should be as fast as possible, since it is invoked during every process switch.
- **Dispatch latency** means the time taken by the dispatcher to
  → stop one process and
  → start another running.

Scheduling Criteria

- Different CPU-scheduling algorithms
  - have different properties and
  - may favor one class of processes over another.
- Criteria to compare CPU-scheduling algorithms:

vtunotesforall.in

1) CPU Utilization
    - We must keep the CPU as busy as possible.
    - In a real system, it ranges from 40% to 90%.
2) Throughput
    - Number of processes completed per time unit.
    - For long processes, throughput may be 1 process per hour; For short transactions, throughput might be 10 processes per second.
3) Turnaround Time
    - The interval from the time of submission of a process to the time of completion.
    - Turnaround time is the sum of the periods spent
    - waiting to get into memory
    - waiting in the ready-queue
    - executing on the CPU and
    - doing I/O.
4) Waiting Time
    - The amount of time that a process spends waiting in the ready-queue.
5) Response Time

¬ The time from the submission of a request until the first response is produced.

¬ The time is generally limited by the speed of the output device.

• We want

→ to maximize CPU utilization and throughput and

→ to minimize turnaround time, waiting time and response time.

Scheduling Algorithms

- CPU scheduling deals with the problem of deciding which of the processes in the ready-queue is to be allocated the CPU.
- Following are some scheduling algorithms:

1) FCFS scheduling (First Come First Served)

2) Round Robin scheduling

3) SJF scheduling (Shortest Job First)

vtunotesforall.in

4) SRT scheduling

5) Priority scheduling

6) Multilevel Queue scheduling and

7) Multilevel Feedback Queue scheduling

FCFS Scheduling

- The process that requests the CPU first is allocated the CPU first.
- The implementation is easily done using a FIFO queue.
- Procedure:

1) When a process enters the ready-queue, its PCB is linked onto the tail of the queue.

2) When the CPU is free, the CPU is allocated to the process at the queue's head.

3) The running process is then removed from the queue.

- Advantage:

1) Code is simple to write & understand.

- Disadvantages:

1) **Convoy effect:** All other processes wait for one big process to get off the CPU.
2) Non-preemptive (a process keeps the CPU until it releases it).
3) Not good for time-sharing systems.
4) The average waiting time is generally not minimal.

- Example: Suppose that the processes arrive in the order P1, P2, P3.

Process	Burst Time
P_1	24
P_2	3
P_3	3

- The Gantt Chart for the schedule is as follows



- Waiting time for P1 = 0; P2 = 24; P3 = 27

vtunotesforall.in

- Average waiting time: (0 + 24 + 27)/3 = 17
- Suppose that the processes arrive in the order P2, P3, P1.
- The Gantt chart for the schedule is as follows:



- Waiting time for P1 = 6;P2 = 0; P3 = 3
- Average waiting time: (6 + 0 + 3)/3 = 3

SJF Scheduling

- The CPU is assigned to the process that has the smallest next CPU burst.
- If two processes have the same length CPU burst, FCFS scheduling is used to break the tie.
- For long-term scheduling in a batch system, we can use the process time limit specified by the user, as the _length'
- SJF can't be implemented at the level of short-term scheduling, because there is no way to know the length of the next CPU burst
- Advantage:
  1) The SJF is optimal, i.e. it gives the minimum average waiting time for a given set of processes.
- Disadvantage:
  1) Determining the length of the next CPU burst.

SJF algorithm may be either 1) non-preemptive or 2) preemptive.

1) Non preemptive SJF

- The current process is allowed to finish its CPU burst.

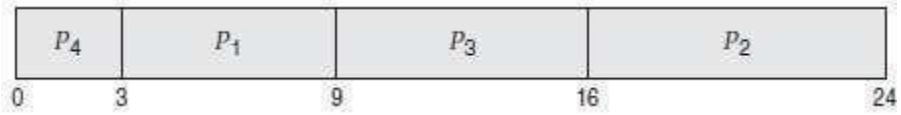
2) Preemptive SJF

- If the new process has a shorter next CPU burst than what is left of the executing process, that process is preempted.
- It is also known as **SRTF** scheduling (Shortest-Remaining-Time-First).

Example (for non-preemptive SJF): Consider the following set of processes, with the length of the CPU-burst time given in milliseconds.

Process	Burst Time
P_1	6
P_2	8
P_3	7
P_4	3

- For non-preemptive SJF, the Gantt Chart is as follows:

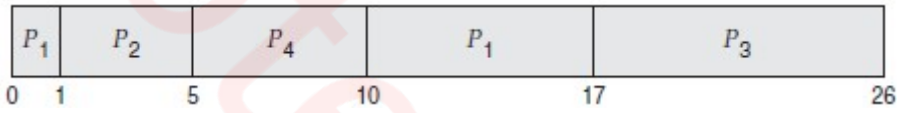


- Waiting time for P1 = 3; P2 = 16; P3 = 9; P4=0
- Average waiting time: (3 + 16 + 9 + 0)/4 = 7

Example (preemptive SJF): Consider the following set of processes, with the length of the CPU-burst time given in milliseconds.

Process	Arrival Time	Burst Time
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

For preemptive SJF, the Gantt Chart is as follows:



The average waiting time is ((10 - 1) + (1 - 1) + (17 - 2) + (5 - 3))/4 = 26/4 = 6.5

Priority Scheduling

- A priority is associated with each process.
- The CPU is allocated to the process with the highest priority.
- Equal-priority processes are scheduled in FCFS order.
- Priorities can be defined either internally or externally.

1) Internally-defined priorities.

- Use some measurable quantity to compute the priority of a process.
- For example: time limits, memory requirements, no. of open files.

2) Externally-defined priorities.

- Set by criteria that are external to the OS
- For example:

→ importance of the process

→ political factors

vtunotesforall.in

Priority scheduling can be either preemptive or nonpreemptive.

1) Preemptive

 The CPU is preempted if the priority of the newly arrived process is higher than the priority of the currently running process.

2) Non Preemptive

 The new process is put at the head of the ready-queue

- Advantage:

1) Higher priority processes can be executed first.

- Disadvantage:

1) Indefinite blocking, where low-priority processes are left waiting indefinitely for CPU. Solution: **Aging** is a technique of increasing priority of processes that wait in system for a long time.

Example: Consider the following set of processes, assumed to have arrived at time 0, in the order PI, P2, ..., P5, with the length of the CPU-burst time given in milliseconds

Process	Burst Time	Priority
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

The Gantt chart for the schedule is as follows:



The average waiting time is 8.2 milliseconds.

Round Robin Scheduling

- Designed especially for timesharing systems.
- It is similar to FCFS scheduling, but with preemption.
- A small unit of time is called a **time quantum** (or time slice).
- Time quantum is ranges from 10 to 100 ms.
- The ready-queueis treated as a **circular queue**.
- The CPU scheduler

→ goes around the ready-queue and

→ allocates the CPU to each process for a time interval of up to 1 time quantum.

- To implement:

  The ready-queue is kept as a FIFO queue of processes

- CPU scheduler

  1) Picks the first process from the ready-queue.

  2) Sets a timer to interrupt after 1 time quantum and

  3) Dispatches the process.

- One of two things will then happen.

  1) The process may have a CPU burst of less than 1 time quantum. In this case, the process itself will release the CPU voluntarily.

  2) If the CPU burst of the currently running process is longer than 1 time quantum, the timer will go off and will cause an interrupt to the OS.

  The process will be put at the tail of the ready-queue.

- Advantage:

  1) Higher average turnaround than SJF.

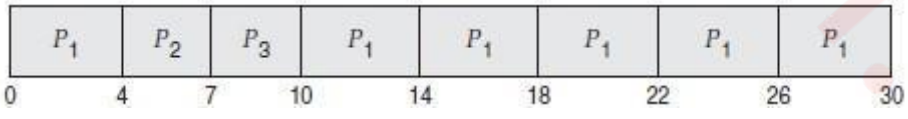
- Disadvantage:

  1) Better response time than SJF.

- Example: Consider the following set of processes that arrive at time 0, with the length of the CPU- burst time given in milliseconds

Process	Burst Time
P_1	24
P_2	3
P_3	3

- The Gantt chart for the schedule is as follows:



The average waiting time is 17/3 = 5.66 milliseconds.

- The RR scheduling algorithm is preemptive.

No process is allocated the CPU for more than 1 time quantum in a row. If a process' CPU burst exceeds 1 time quantum, that process is preempted and is put back in the ready-queue..

- The performance of algorithm depends heavily on the size of the time quantum (Figure 2.8 & 2.9).

1) If time quantum=very large, RR policy is the same as the FCFS policy.

2) If time quantum=very small, RR approach appears to the users as though each of n processes has its own processor running at l/n the speed of the real processor.

- In software, we need to consider the effect of context switching on the performance of RR scheduling

1) Larger the time quantum for a specific process time, less time is spending on context switching.
2) The smaller the time quantum, more overhead is added for the purpose of context-switching.

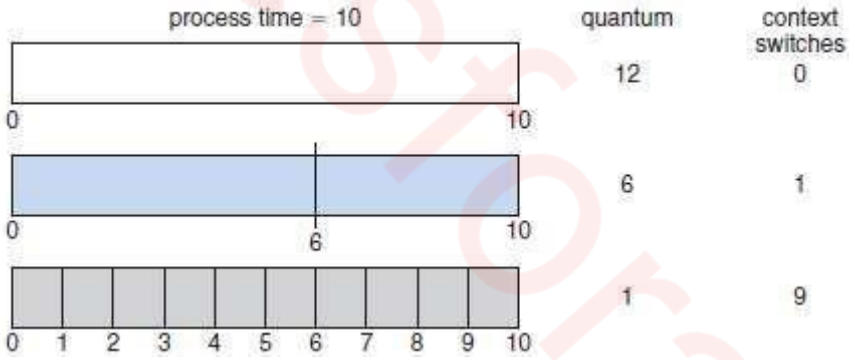
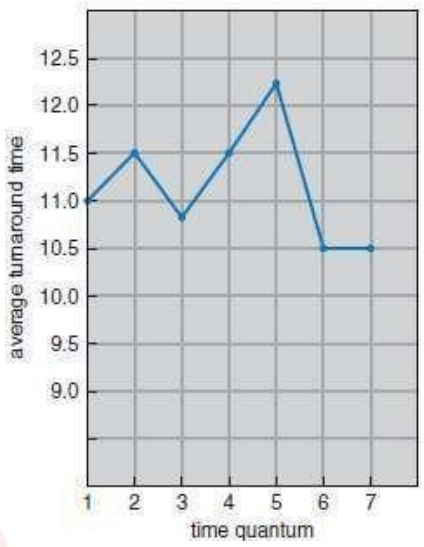


Figure 2.8 How a smaller time quantum increases context switches

vtunotesforall.in



process	time
P_1	6
P_2	3
P_3	1
P_4	7

Figure 2.9 How turnaround time varies with the time quantum

Multilevel Queue Scheduling

- Useful for situations in which processes are easily classified into different groups.
- For example, a common division is made between

  → foreground (or interactive) processes and

  → background (or batch) processes.

- The ready-queueis partitioned into several separate queues (Figure 2.10).
- The processes are permanently assigned to one queue based on some property like

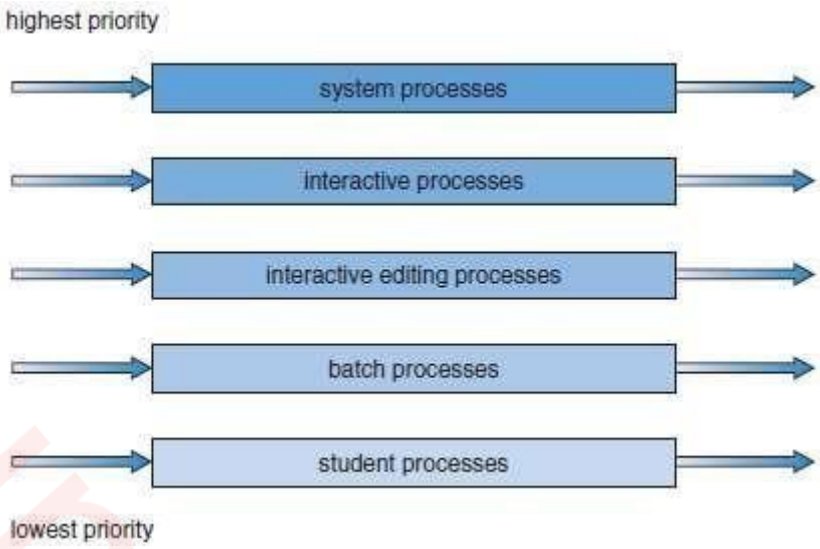
  → memory size

  → process priority or

  → process type.

- Each queue has its own scheduling algorithm.
- For example, separate queues might be used for foreground and background processes.

vtunotesforall.in



- There must be scheduling among the queues, which is commonly implemented as fixed-priority pre-emptive scheduling.
- For example, the foreground queue may have absolute priority over the background queue.
- **Time slice**: each queue gets a certain amount of CPU time which it can schedule amongst its processes; i.e., 80% to foreground in RR 20% to background in FCFS.

Multilevel Feedback Queue Scheduling

- A process may move between queues (Figure 2.11).
- The basic idea:

Separate processes according to the features of their CPU bursts. For example

1) If a process uses too much CPU time, it will be moved to a lower-priority queue.
   ¤ This scheme leaves I/O-bound and interactive processes in the higher-priority queues.
2) If a process waits too long in a lower-priority queue, it may be moved to a higher-priority queue.
   ¤ This form of aging prevents starvation.

vtunotesforall.in

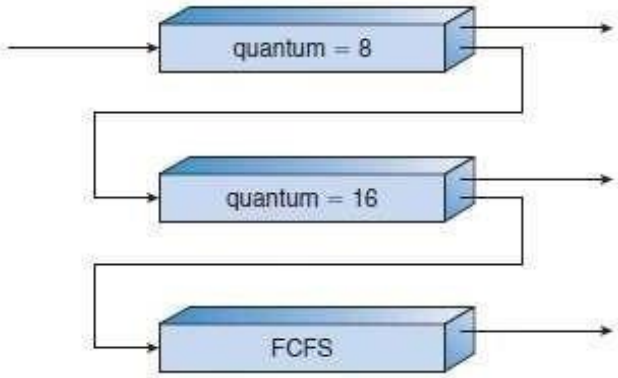


Figure 2.11 Multilevel feedback queues.

- In general, a multilevel feedback queue scheduler is defined by the following parameters:

  1) The number of queues.

  2) The scheduling algorithm for each queue.

  3) The method used to determine when to upgrade a process to a higher priority queue.

  4) The method used to determine when to demote a process to a lower priority queue.

  5) The method used to determine which queue a process will enter when that process needs service.

Multiple Processor Scheduling

- If multiple CPUs are available, the scheduling problem becomes more complex.
- Two approaches:

  1) Asymmetric Multiprocessing

  - The basic idea is:

    i) A master server is a single processor responsible for all scheduling decisions, I/O processing and other system activities.

    ii) The other processors execute only user code.

  - Advantage:

    i) This is simple because only one processor accesses the system data structures, reducing the need for data sharing.

  2) Symmetric Multiprocessing

  - The basic idea is:

    i) Each processor is self-scheduling.

    ii) To do scheduling, the scheduler for each processor

      i. Examines the ready queue and

vtunotesforall.in

ii. Selects a process to execute.

 Restriction: We must ensure that two processors do not choose the same process and that processes are not lost from the queue.

Processor Affinity

- In SMP systems,

1) Migration of processes from one processor to another are avoided and

2) Instead processes are kept running on same processor. This is known as processor affinity.

- Two forms:

1) Soft Affinity

 When an OS try to keep a process on one processor because of policy, but cannot guarantee it will happen.

 It is possible for a process to migrate between processors.

2) Hard Affinity

 When an OS have the ability to allow a process to specify that it is not to migrate to other processors. Eg: Solaris OS.

Load Balancing

- This attempts to keep the workload evenly distributed across all processors in an SMP system.
- Two approaches:

1) Push Migration

 A specific task periodically checks the load on each processor and if it finds an imbalance, it evenly distributes the load to idle processors.

2) Pull Migration

 An idle processor pulls a waiting task from a busy processor.

Symmetric Multithreading

- The basic idea:

1) Create multiple logical processors on the same physical processor.

vtunotesforall.in

2) Present a view of several logical processors to the OS.

- Each logical processor has its own architecture state, which includes general -purpose and machine- state registers.
- Each logical processor is responsible for its own interrupt handling.
- SMT is a feature provided in hardware, not software.

Thread Scheduling

- On OSs, it is kernel-level threads but not processes that are being scheduled by the OS.
- User-level threads are managed by a thread library, and the kernel is unaware of them.
- To run on a CPU, user-level threads must be mapped to an associated kernel-level thread.

Contention Scope

- Two approaches:

1) Process-Contention scope

 On systems implementing the many-to-one and many-to-many models, the thread library schedules user-level threads to run on an available LWP.

 Competition for the CPU takes place among threads belonging to the same process.

2) System-Contention scope

 The process of deciding which kernel thread to schedule on the CPU.

 Competition for the CPU takes place among all threads in the system.

 Systems using the one-to-one model schedule threads using only SCS.

Pthread Scheduling

- Pthread API that allows specifying either PCS or SCS during thread creation.
- Pthreads identifies the following contention scope values:

1) PTHREAD_SCOPEJPROCESS schedules threads using PCS scheduling.

2) PTHREAD-SCOPE_SYSTEM schedules threads using SCS scheduling.

- Pthread IPC provides following two functions for getting and setting the contention scope policy:

1) pthread_attr_setscope(pthread_attr_t *attr, int scope)

2) pthread_attr_getscope(pthread_attr_t *attr, int *scope)

Exercise Problems

1) Consider the following set of processes, with length of the CPU burst time given in milliseconds:

Process	Arrival Time	Burst Time	Priority
P1	0	10	3
P2	0	1	1
P3	3	2	3
P4	5	1	4
P5	10	5	2

(i) Draw four Gantt charts illustrating the execution of these processing using FCFS, SJF, a non preemptive priority and RR (Quantum=2) scheduling.

(ii) What is the turn around time of each process for each scheduling algorithm in (i).

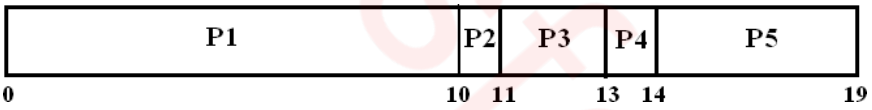
(iii) What is waiting time of each process in (i)

Solution:

$$\text{Average turn around time} = \frac{\text{Sum of waiting time of individual process}}{\text{Number of processes}}$$

$$\text{Average waiting time} = \frac{\text{Sum of turn around time of individual process}}{\text{Number of processes}}$$

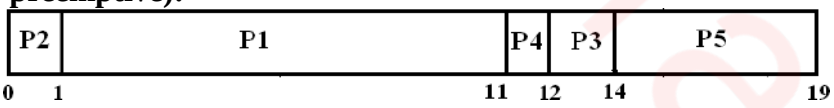
(i) FCFS:



Average waiting time = (0+10+8+8+4)/5 = 6

Average turnaround time = (10+11+13+14+19)/5 = 13.4

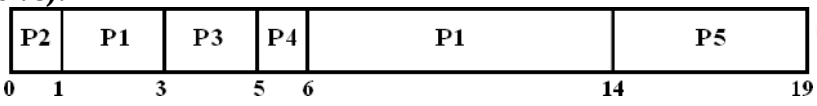
(ii) SJF (non-preemptive):



Average waiting time = (1+0+9+6+4)/5 = 4

Average turnaround time = (11+1+14+12+19)/5 = 11.4

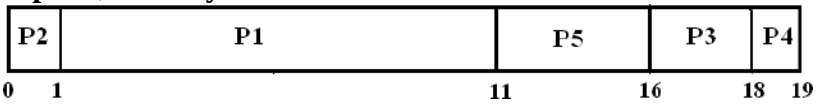
SJF (preemptive):



Average waiting time = (4+0+0+0+4)/5 = 1.6

Average turnaround time = (14+1+5+6+19)/5 = 9

(iii) Non-preemptive, Priority:

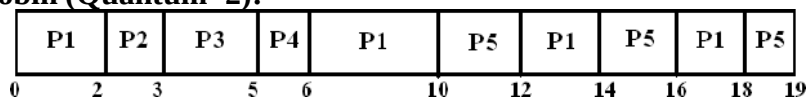


vtunotes.org.in

Average waiting time = (1+0+13+13+1)/5 = 5.6

Average turnaround time = (11+1+18+19+16)/5 = 13

(iv) Round Robin (Quantum=2):



Average waiting time = (8+2+0+0+4)/5 = 2.8

Average turnaround time = (18+3+5+6+19)/5 = 10.2

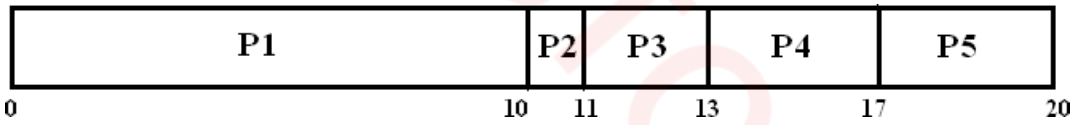
2) Consider the following set of process with arrival time:

i) Draw grant chart using FCFS, SJF preemptive and non preemptive scheduling.

ii) Calculate the average waiting and turnaround time for each process of the scheduling algorithm.

Process	Arrival Time	Burst
P1	0	10
P2	0	1
P3	1	2
P4	2	4
P5	2	3

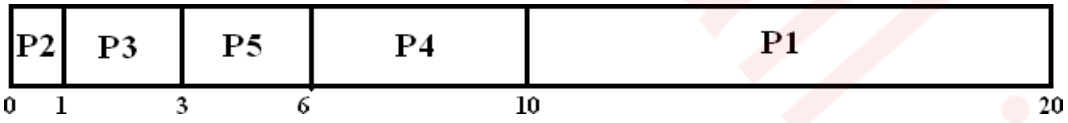
Solution: (i) FCFS:



Average waiting time = (0+10+10+11+15)/5 = 9.2

Average turnaround time = (10+11+13+17+20)/5 = 14.2

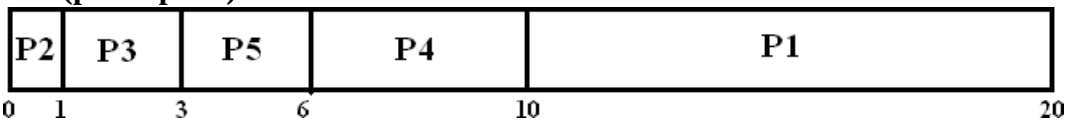
(ii) SJF (non-preemptive):



Average waiting time = (10+0+0+4+1)/5 = 3

Average turnaround time = (20+1+3+10+6)/5 = 8

(iii) SJF (preemptive):



Average waiting time = (10+0+0+4+1)/5 = 3

Average turnaround time = (20+1+3+10+6)/5 = 8

3) Consider following set of processes with CPU burst time (in msec)

Process	Arrival Time	Burst Time
P0	0	6
P1	1	3
P2	2	1
P3	3	4

i) Draw Gantt chart illustrating the execution of above processes using SRTF and non preemptive SJF

ii) Find the turnaround time for each process for SRTF and SJF. Hence show that SRTF is faster than

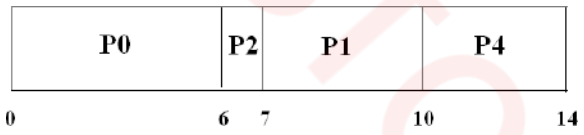
SJF.

Solution:

$$\text{Average turn around time} = \frac{\text{Sum of waiting time of individual process}}{\text{Number of processes}}$$

$$\text{Average waiting time} = \frac{\text{Sum of turn around time of individual process}}{\text{Number of processes}}$$

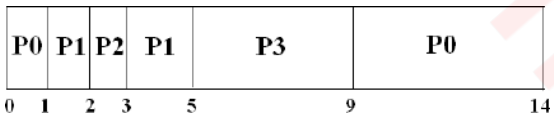
(i) Non-preemptive SJF:



Average waiting time = (0+6+4+7)/4 = 4.25

Average turnaround time = (6+10+7+14)/4 = 9.25

(ii) SRTF (preemptive SJF):



Average waiting time = (8+1+0+2)/4 = 2.75

Average turnaround time = (14+5+3+9)/4 = 7.75

Conclusion:

Since average turnaround time of SRTF(7.75) is less than SJF(9.25), SRTF is faster than SJF

vtunotesforall.in

4) Following is the snapshot of a cpu

Process	Arrival Time	Burst Time
P1	0	10
P2	1	29
P3	2	03
P4	3	07

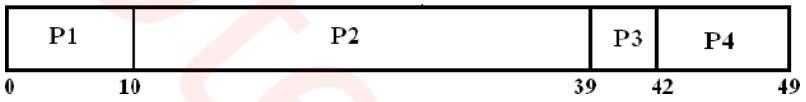
Draw Gantt charts and calculate the waiting and turnaround time using FCFS, SJF and RR with time quantum 10 scheduling algorithms.

Solution:

$$\text{Average turn around time} = \frac{\text{Sum of waiting time of individual process}}{\text{Number of processes}}$$

$$\text{Average waiting time} = \frac{\text{Sum of turn around time of individual process}}{\text{Number of processes}}$$

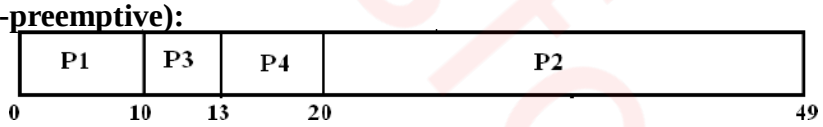
(i) FCFS:



Average waiting time = (0+9+37+39)/4 = 21.25

Average turnaround time = (10+39+42+49)/4= 35

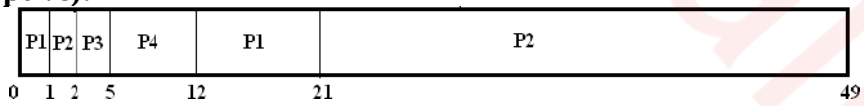
(ii) SJF (non-preemptive):



Average waiting time = (0+19+8+10)/4 = 9.25

Average turnaround time = (10+49+13+20)/4 = 19

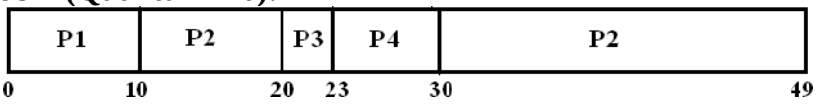
SJF (preemptive):



Average waiting time = (11+19+0+2)/4 = 8

Average turnaround time = (21+49+5+12)/4 = 21.75

(iii) Round Robin (Quantum=10):



vtunotesforall.in

Average waiting time = (0+19+18+20)/4 = 14.25

Average turnaround time = (10+49+23+30)/4 = 28

5. Consider the following snapshot

Process	Arrival Time	Burst Time
P1	0	5
P2	1	1
P3	2	4

Compute average turnaround time and average waiting time using

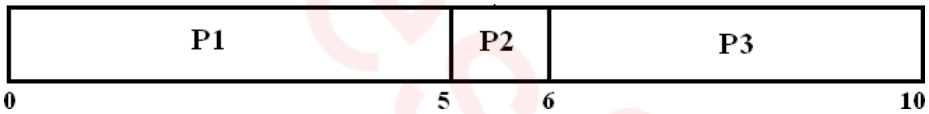
i) FCFS
ii) Preemptive SJF and
iii) iii) RR (quantum-4).

Solution:

$$\text{Average turn around time} = \frac{\text{Sum of waiting time of individual process}}{\text{Number of processes}}$$

$$\text{Average waiting time} = \frac{\text{Sum of turn around time of individual process}}{\text{Number of processes}}$$

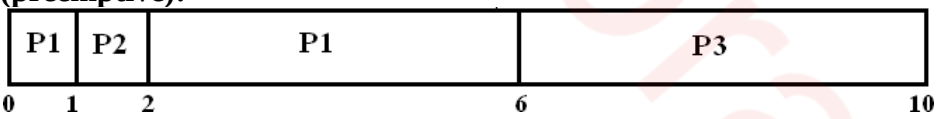
(i) FCFS:



Average waiting time = (0+4+4)/3 = 2.67

Average turnaround time = (5+6+10)/3 = 6.67

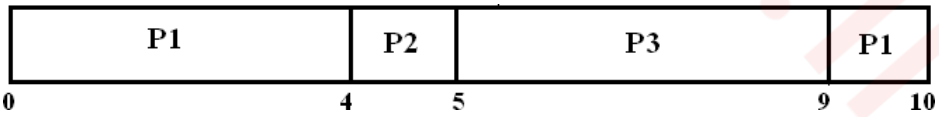
(ii) SJF (preemptive):



Average waiting time = (1+0+4)/3 = 1.67

Average turnaround time = (6+2+10)/3 = 6

(iii) Round Robin (Quantum=4):



Average waiting time = (5+3+3)/3 = 3.34

Average turnaround time = (10+5+9)/3 = 8

6. Consider the following set of processes, with arrival time and burst time in ms.

Process	Arrival Time	Priority	Burst Time
P1	0 ms	3	3 ms
P2	1 ms	2	4 ms
P3	2 ms	4	6 ms
P4	3 ms	6	4 ms
P5	5 ms	10	2 ms

Compute average turnaround time and average waiting time using

i) FCFS
ii) Preemptive SJF and
iii) Non preemptive SJF
iv) RR (quantum-2).
v) Non Preemptive priority scheduling
vi) Preemptive priority scheduling.

7. Consider the following set of processes, with arrival time and burst time in ms.

Process Id	Arrival time	Burst time	Priority
P1	0	4	2
P2	1	3	3
P3	2	1	4
P4	3	5	5
P5	4	2	5

Compute average turnaround time and average waiting time using

i) FCFS
ii) Preemptive SJF and
iii) Non preemptive SJF
iv) RR (quantum-2).
v) Non Preemptive priority scheduling
vi) Preemptive priority scheduling.

vtunotesforall.in

8. Consider the following set of processes, with arrival time and burst time in ms

Process ID	Arrival Time (milliseconds)	Burst Time (milliseconds)	Priority number
P1	0	4	3
P2	1	2	2
P3	2	3	4
P4	4	2	1

Compute average turnaround time and average waiting time using

i) FCFS
ii) Preemptive SJF and
iii) Non preemptive SJF
iv) RR (quantum-2).
v) Non Preemptive priority scheduling
vi) Preemptive priority scheduling

vtunotesforall.in